

PROJECT REPORT

Banking Information System

Core Java Console Application

Project Title	Banking Information System
Platform	Core Java — Console Application
Author	Pawan Kumar V
Organization	Upskill Campus — Core Java Internship
Language	Java (JDK 11+)
Source File	BankingInformationSystem.java
Total Lines	~332 lines
Report Date	5 June 2026

1. Project Overview

The Banking Information System is a console-based Java application developed as part of the Upskill Campus Core Java Internship. It simulates essential banking operations within a single Java source file, demonstrating the practical application of object-oriented programming, data structures, and user-interaction design in a terminal environment.

The system supports multiple user accounts stored in memory, each capable of performing real-world banking tasks such as deposits, withdrawals, fund transfers, and statement retrieval — all secured by password authentication.

2. Objectives

- Design and implement a working banking prototype using Core Java.
- Apply object-oriented principles through inner static classes (BankAccount, Transaction).
- Demonstrate use of Java Collections (HashMap, ArrayList) for in-memory data management.
- Implement user authentication with account number and password verification.
- Handle edge cases: insufficient funds, duplicate transfers to self, invalid inputs.
- Provide a clean, menu-driven console UI with formatted output.

3. Technology Stack

Language	Java SE (JDK 11 or higher)
Collections	java.util.HashMap, java.util.ArrayList
I/O	java.util.Scanner (standard input)
Date & Time	java.util.Date, java.text.SimpleDateFormat
IDE / Compiler	Any standard JDK (javac / java CLI)
Storage	In-memory (runtime only, no database/file I/O)
Design Pattern	Procedural with inner static class models

4. System Architecture

The entire application is contained in one public class `BankingInformationSystem`. It uses two inner static classes as data models and a set of static methods as the service layer. Global static state holds the account registry and the currently logged-in session.

4.1 Class Structure

<code>BankingInformationSystem</code>	Main class — entry point, menus, all business logic
<code>BankingInformationSystem.Transaction</code>	Inner model — stores transaction type, amount, balance, date, description
<code>BankingInformationSystem.BankAccount</code>	Inner model — stores account holder info, password, balance, transaction list

4.2 Global State Variables

<code>accounts</code>	<code>HashMap<String, BankAccount></code> — maps account number → account object
<code>loggedInAccount</code>	<code>BankAccount</code> — reference to the currently authenticated session
<code>scanner</code>	<code>Scanner</code> — single shared input stream throughout runtime
<code>accountCounter</code>	<code>int</code> — auto-incrementing counter, starts at 1001 for account numbering

5. Module & Method Breakdown

Module/Method	Lines	Description
<code>main()</code>	~25	Application entry point. Prints banner and drives the main menu loop (Register / Login / Exit).
<code>registerUser()</code>	~28	Collects user details, validates password match/length, accepts initial deposit, generates account number, creates <code>BankAccount</code> and stores in <code>HashMap</code> .
<code>login()</code>	~13	Looks up account by number, verifies password, sets <code>loggedInAccount</code> on success.
<code>dashboard()</code>	~27	Post-login menu loop. Routes to all banking operations. Clears session on logout.
<code>deposit()</code>	~13	Reads deposit amount, validates > 0 , updates balance, appends CREDIT transaction.
<code>withdraw()</code>	~17	Reads withdrawal amount, checks insufficient funds, updates balance, appends DEBIT transaction.

fundTransfer()	~38	Validates recipient account exists, prevents self-transfer, checks balance, updates both accounts, appends DEBIT/CREDIT transactions on each side.
accountStatement()	~22	Prints formatted transaction history: date, type, amount, running balance, description.
viewAccount()	~9	Displays all stored profile fields for the logged-in account.
updateAccount()	~15	Allows partial updates to name, address, phone, email — blank input preserves existing value.
generateAccountNumber()	~3	Returns 'ACC' + auto-incremented counter.
readLine()	~4	Utility: prompts and reads a trimmed string.
readDouble()	~9	Utility: loops until a valid non-negative number is entered.
printHeader()	~5	Utility: draws a Unicode box header with the given title.
printLine()	~3	Utility: prints a horizontal rule.

6. Data Flow & Logic

6.1 Registration Flow

- User provides: full name, address, phone, email, password (confirmed), initial deposit.
- Password must match confirmation and be at least 4 characters.
- A unique account number is generated (ACC1001, ACC1002, ...).
- A new BankAccount is instantiated. If initial deposit > 0, an 'Initial Deposit' CREDIT transaction is auto-recorded.
- Account is stored in the accounts HashMap.

6.2 Login Flow

- User enters account number and password.
- HashMap lookup retrieves the BankAccount by account number.
- If found and password matches, loggedInAccount is set and dashboard() is invoked.

6.3 Transaction Flow (Deposit / Withdraw / Transfer)

- Amount is validated (> 0 and, for withdrawal/transfer, ≤ available balance).
- Balance is updated in-memory directly on the BankAccount object.
- A Transaction object (type, amount, running balance, description, timestamp) is appended to the account's transaction list.
- For fund transfers, both sender and recipient accounts are updated atomically in the same method call.

7. Features & Functionality

Account Registration	Unique auto-generated account number, password with minimum length enforcement.
Secure Login	Account number + password authentication; failed login shows error without detail.
Deposit	Adds funds; records CREDIT transaction with timestamp.
Withdrawal	Deducts funds; validates against available balance; records DEBIT transaction.
Fund Transfer	Bidirectional: debits sender, credits recipient; prevents self-transfer.
Account Statement	Tabular display of all transactions with date, type, amount, and running balance.
View Account Details	Displays profile: name, address, phone, email, balance.
Update Account Info	Partial update support — blank input retains the current value.

Input Validation	Non-numeric input for amounts is caught; negative amounts blocked; self-transfer blocked.
Formatted Console UI	Unicode box-drawing characters create a structured, readable terminal interface.

8. Error Handling

Insufficient Funds	Withdrawal and transfer check balance before deducting; user sees '[!] Insufficient funds.'
Invalid Amount Format	readDouble() catches NumberFormatException and prompts again in a loop.
Negative Amount	readDouble() rejects values < 0 immediately.
Zero Amount	deposit() and withdraw() explicitly reject amount == 0.
Wrong Credentials	login() returns false with a generic error; no detail on which field is wrong.
Recipient Not Found	fundTransfer() checks accounts.get(); shows '[!] Recipient account not found.'
Self-Transfer	fundTransfer() compares account numbers and blocks the operation.
Password Mismatch	registerUser() loops until password and confirmation match and meet length requirement.

9. Sample Execution Flow

Below is a representative walkthrough of a typical session:

Step 1 – Launch: The banner is displayed with the main menu (Register / Login / Exit).

Step 2 – Register: User selects [1], enters details, receives 'ACC1001'.

Step 3 – Login: User selects [2], enters ACC1001 + password, reaches dashboard.

Step 4 – Deposit ₹5000: Balance updates from ₹1000 (initial) to ₹6000.

Step 5 – Transfer ₹2000 to ACC1002: Sender at ₹4000, recipient at ₹2000+.

Step 6 – Statement: Full transaction history with running balances.

10. Limitations & Future Improvements

10.1 Current Limitations

- All data is stored in-memory; all accounts and transactions are lost when the program exits.
- Passwords are stored as plain strings — no hashing or encryption.
- No file or database persistence layer.
- No concurrent session support; single-threaded, single-user at a time.
- Account numbers are sequential integers with no check-digit or branch code.

10.2 Suggested Enhancements

- Integrate file I/O (Java Serialization or JSON) or a lightweight database (SQLite via JDBC) for data persistence.
- Implement password hashing using MessageDigest (SHA-256) or BCrypt.
- Add a transaction ID and a debit/credit reference number per transaction.
- Introduce interest calculation for savings accounts.
- Build a JavaFX or Swing GUI to replace the console interface.
- Add a mini account statement filter (date range, type).
- Support multiple account types: Savings, Current, Fixed Deposit.

11. Conclusion

The Banking Information System successfully demonstrates a broad range of Core Java fundamentals in a practical, real-world context. From OOP design with inner static classes to data management with HashMap and ArrayList, from input validation loops to formatted console output, the project covers the essential competencies of the Upskill Campus Core Java curriculum.

While intentionally scoped to an in-memory prototype without persistence or encryption, the codebase provides a clear, well-organised foundation that can be extended into a production-grade application.

The project reflects good separation of concerns — models, utilities, and business logic are cleanly divided across static methods — making it an effective learning artefact and a strong internship deliverable.

— *Pawan Kumar V, Upskill Campus Core Java Internship*